

Building Intelligent Agents with LangChain

A Hands-On Tutorial | Agentic AI Deep Dive

In collaboration with IIT Delhi

Agentic AI

LangChain

Hands-On

What is Agentic AI?

Agentic AI is artificial intelligence that can reason about goals, decide on actions, use tools, and iterate autonomously — without human intervention at every step.

Thinks & Reasons

Analyzes the problem and plans a series of steps to solve it.

Acts & Executes

Calls tools like web search, calculators, APIs to take action.

Observes & Learns

Evaluates the result of each action and decides if more is needed.

Repeats Until Done

Loops back and keeps going until the goal is fully achieved.

Key Insight: Traditional AI answers questions. Agentic AI solves multi-step problems autonomously.

Why Agentic AI Now?

Three key advances happening simultaneously make agentic AI possible today:

01

LLMs Are Now Smart Enough

Models like GPT-4 and Claude can reason over complex multi-step problems, follow long instructions, and decide what to do next — without hand-holding.

02

Tool Calling Is Real

LLMs can now invoke actual functions: search the web, query databases, call APIs, do calculations. They don't just generate text — they take action.

03

Businesses Demand Automation

Companies need AI that doesn't just chat. They want end-to-end automation of workflows — research, analysis, reporting, decision-making — without manual steps.

Real-world agents today: AutoGPT | Devin (AI Developer) | Perplexity | GitHub Copilot Workspace | ChatGPT with Web

Where is Agentic AI Being Used?

Finance

- Automated market research & reporting
- Portfolio risk assessment
- Fraud detection pipelines
- Regulatory document summarization

Healthcare

- Clinical trial patient matching
- Medical document analysis
- Diagnosis support from records
- Drug interaction research

Software

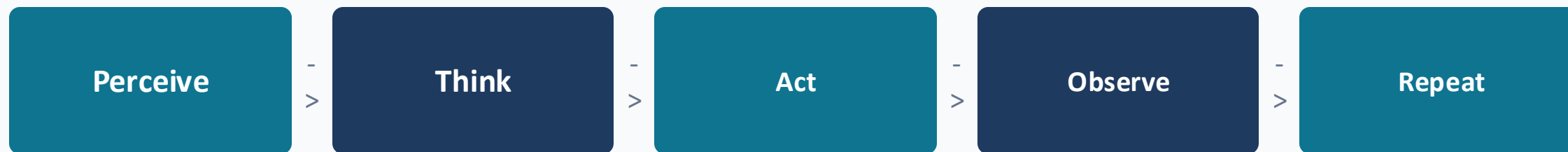
- AI pair programming (GitHub Copilot)
- Automated bug detection & fixing
- Test case generation
- Code review & refactoring

Enterprise Ops

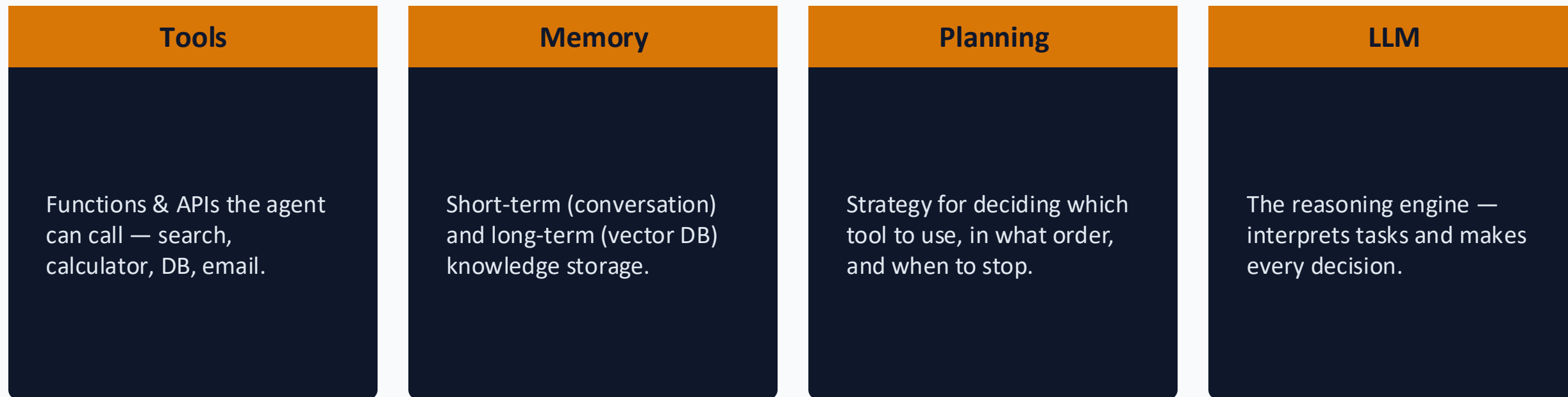
- Workflow automation (approvals, emails)
- Internal Q&A over documents
- Employee onboarding automation
- Knowledge discovery at scale

The Agentic AI Framework

Every agent — regardless of complexity — follows the same core loop:



Built on four core pillars:



Enter LangChain

The open-source Python framework that makes building agentic AI dramatically easier.

Open Source & Free

MIT licensed, maintained by a large community with 800+ integrations across every AI tool.

LLM Agnostic

Swap between OpenAI, Anthropic Claude, Google Gemini, Meta Llama, or any local model easily.

Built for Agents

Pre-built agent types, tool decorators, memory managers — specifically designed for agentic workflows.

Active Ecosystem

LangChain Hub, LangGraph, and a massive community with docs, tutorials, and Discord support.

LangChain Key Components: Foundation

These three form the foundation — every LangChain application is built from them:

LLMs / Chat Models

The intelligence layer of your agent. You choose which model powers the reasoning — GPT-4, Claude, Gemini, or open-source alternatives like Llama. LangChain wraps all of them with one unified interface.

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model='gpt-4', temperature=0)
```

Prompts & Templates

Structured, reusable instructions sent to the LLM. Templates let you inject dynamic variables — the user's query, available tools, retrieved documents. You define the format once and reuse it everywhere.

```
from langchain.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([...])
```

Chains

A sequence of components connected together. The most common pattern: Prompt -> LLM -> Output Parser. LangChain's pipe operator (|) lets you compose chains cleanly, and you can nest chains inside other chains.

```
chain = prompt | llm | output_parser
result = chain.invoke({'question': user_query})
```

LangChain Key Components: The Agent Stack

Layer these on top of the foundation to get a fully autonomous agent:

Tools

Functions the agent can call. Can be anything: a web search API, a calculator function, a database query, an email sender. You define them with the `@tool` decorator and LangChain handles the rest.

```
from langchain.tools import tool
@tool
def search(query: str) -> str:
    return web_search(query)
```

Memory

Gives the agent context. Short-term memory stores the conversation history so the agent remembers what was said. Long-term memory uses vector databases so the agent can retrieve relevant knowledge from large document collections.

```
from langchain.memory import
ConversationBufferMemory
memory = ConversationBufferMemory()
memory.save_context({'input': q}, {'output': a})
```

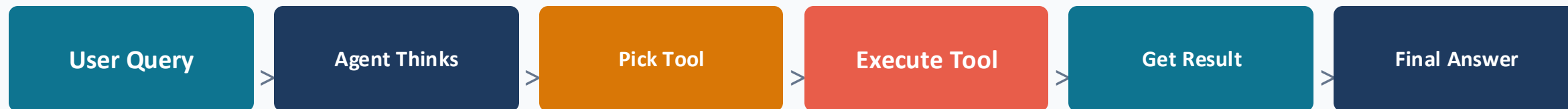
Agents & AgentExecutor

The orchestrator. Decides which tool to call, with what input, and in what order. The ReAct agent (most common) alternates between reasoning about the problem and acting by calling tools, until the task is complete.

```
from langchain.agents import AgentExecutor,
create_react_agent
agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools)
```


The Agent Loop: ReAct Pattern

ReAct = Reasoning + Acting. The LLM alternates between thinking and calling tools until the task is done.



How the loop works, step by step:

1. User submits a question or task to the agent.
2. Agent reasons: 'What do I need to do? Which tool helps?'
3. Agent picks the right tool and forms the input for it.
4. Tool executes — runs a web search, does a calculation, etc.
5. Agent receives the result and evaluates: 'Is this enough?'
6. If yes, the agent writes a final answer. If no, it loops back to step 2.

Pseudocode

```
while not task_complete:

    # Step 1: LLM reasons about the task
    thoughts = agent.think(
        history + available_tools
    )

    # Step 2: LLM decides which tool
    action, action_input = \
        agent.choose_tool(thoughts)

    # Step 3: Tool executes
    observation = tools[action].run(
        action_input
    )

    # Step 4: Append result, loop back
    history.append(observation)
```

What We'll Build Today

A working ReAct agent with real tools, explained line by line.

01

**A ReAct Agent with Two
Tools**

02

Step-by-Step Code
Walkthrough

03

**Live Execution &
Observation**

Ready. Let's code.